
Pre-Read for the Masterclass

Scaler Academy · NexaML

This pre-read prepares you for a 2.5-hour deep-dive session on building production-grade Retrieval-Augmented Generation systems. By the time you finish this document you will understand **why naive RAG breaks at scale**, what the key architectural components are, and which design choices matter most in practice.

■ Reading Time	■ Sections	■ Target Audience	■ Prerequisites
~25 minutes	6 Sections	MLE / DS with LLM basics	Python + Embeddings 101

#	Section
1	Why RAG? — The Problem with Pure LLMs
2	Naive RAG — The First Solution & Its Limits
3	Failure Modes at Scale
4	Scalable RAG — The Full Architecture Blueprint
5	Key Components Deep Dive
6	Production Checklist & What to Expect in Class

Section 1 — Why RAG? The Problem with Pure LLMs

The Analogy: Imagine you hired the world's smartest consultant. She read every book, paper, and article ever written — but she stopped reading 18 months ago. Now you ask her: *What are our Q3 FY25 product returns in the North region?* She has no idea. She cannot look at your company's data. She cannot browse the web. She will either say *I don't know* or — worse — **make something up**. That consultant is a pure LLM.

The Three Core Problems

- **Knowledge Cutoff** — LLMs are trained on a frozen snapshot of the world. Anything after that date is invisible to them. GPT-4's cutoff is April 2023; Claude Sonnet's is early 2024. Your business data, today's news, your internal policy docs — all invisible.
- **Hallucination** — When an LLM doesn't know an answer, it doesn't say 'I don't know'. It *generates* a plausible-sounding answer. In a customer support bot or a legal document tool, this is catastrophic. The model can confidently cite a court case that doesn't exist.
- **No Private Data Access** — The LLM has never seen your company's Confluence wiki, your Salesforce notes, your Slack threads, or your product PDFs. Fine-tuning is expensive, slow, and doesn't scale. You need a smarter approach.

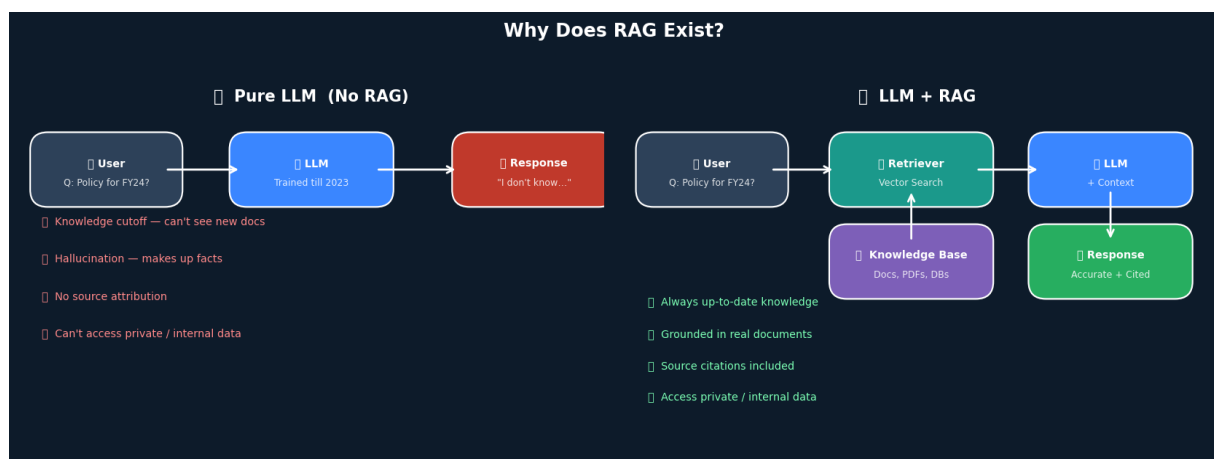


Figure 1: Pure LLM vs RAG-augmented LLM — the same query, very different outcomes.

■ **The RAG Insight:** Instead of forcing the LLM to memorise everything, let it be a reasoning engine. At query time, retrieve the relevant documents from an external store and inject them into the LLM's context window. The LLM now reasons over fresh, specific, grounded information.

Section 2 — Naive RAG: The First Solution

Naive RAG (also called 'vanilla RAG') is the simplest working implementation. It has two distinct phases — an offline **indexing pipeline** that runs once (or periodically), and an online **query pipeline** that runs on every user request.

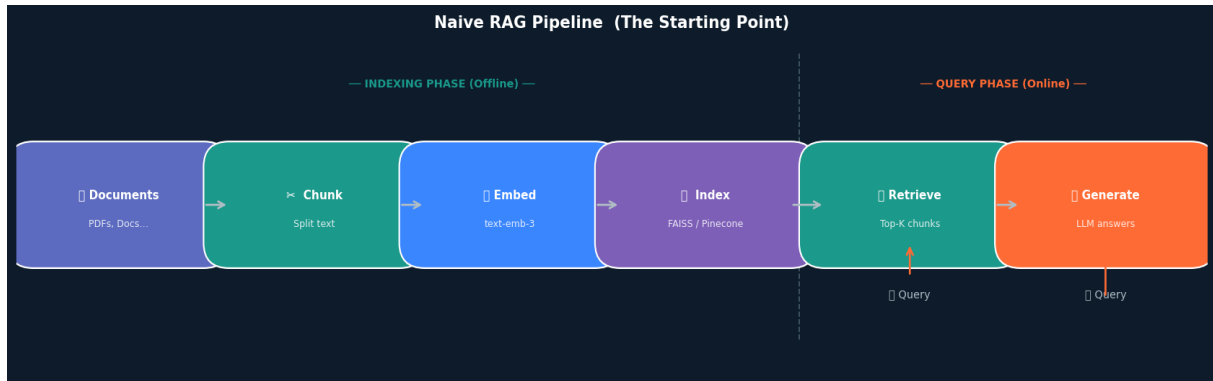


Figure 2: The Naive RAG pipeline — two phases, six steps.

Step-by-Step Walkthrough

- 1. Load Documents:** Ingest raw files — PDFs, Word docs, HTML pages, database tables. Use loaders like LangChain's PyPDFLoader or Unstructured.io.
- 2. Chunk:** Split long documents into smaller pieces. A typical naive setting: `chunk_size=512` tokens, `chunk_overlap=50` tokens.
- 3. Embed:** Pass each chunk through an embedding model (e.g. OpenAI text-embedding-3-small, or open-source BGE-M3). Each chunk becomes a dense vector of 768–1536 dimensions.
- 4. Index:** Store vectors in a vector database: FAISS (in-memory), Pinecone, Qdrant, Weaviate, or pgvector. This is your 'semantic search engine'.
- 5. Retrieve:** At query time, embed the user's question and do a nearest-neighbour search in the vector store. Return the Top-K most similar chunks (typically $K=3-5$).
- 6. Generate:** Assemble a prompt: [System: You are a helpful assistant...] + [Context:] + [Question:] → send to LLM → return answer.

■ ■ **Where Naive RAG Works Well:** Small document sets (<10K chunks), simple factual queries, internal demos, and proof-of-concepts. It fails when you push it to production. Let's see why.

Section 3 — Failure Modes at Scale

When Swiggy engineers built their internal ops assistant, the naive RAG prototype worked fine on 500 documents. At 2 million documents and 10,000 daily queries, it collapsed. Here's why:

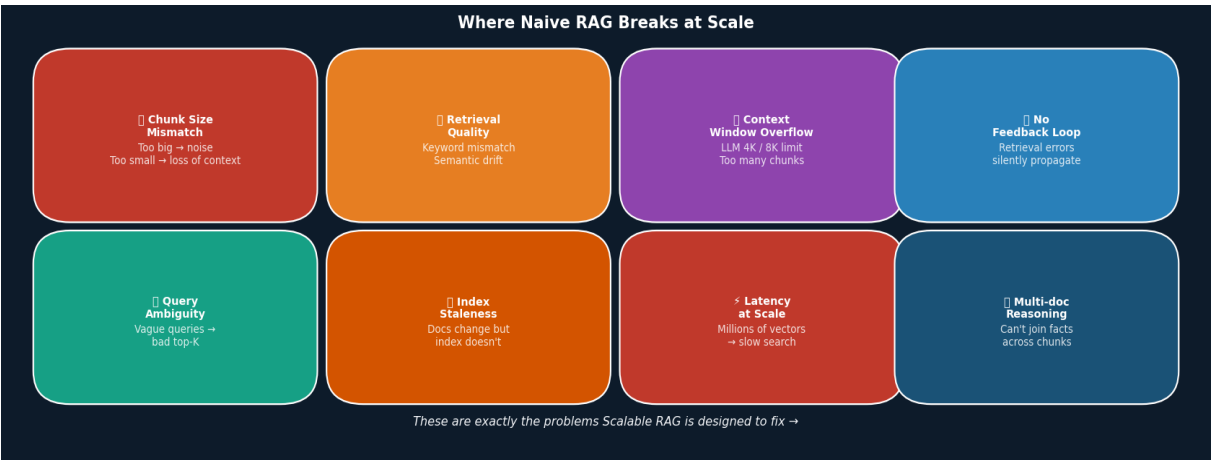


Figure 3: Eight failure modes that naive RAG cannot handle at scale.

Section 4 — The Scalable RAG Architecture

Scalable RAG adds four key pillars on top of naive RAG: a **robust ingestion pipeline** that handles diverse, large-scale data sources; a **smarter query pipeline** with query rewriting, hybrid retrieval, and re-ranking; an **evaluation + feedback loop** to continuously measure and improve retrieval quality; and a **production infrastructure layer** covering caching, async queues, auth, and model serving.

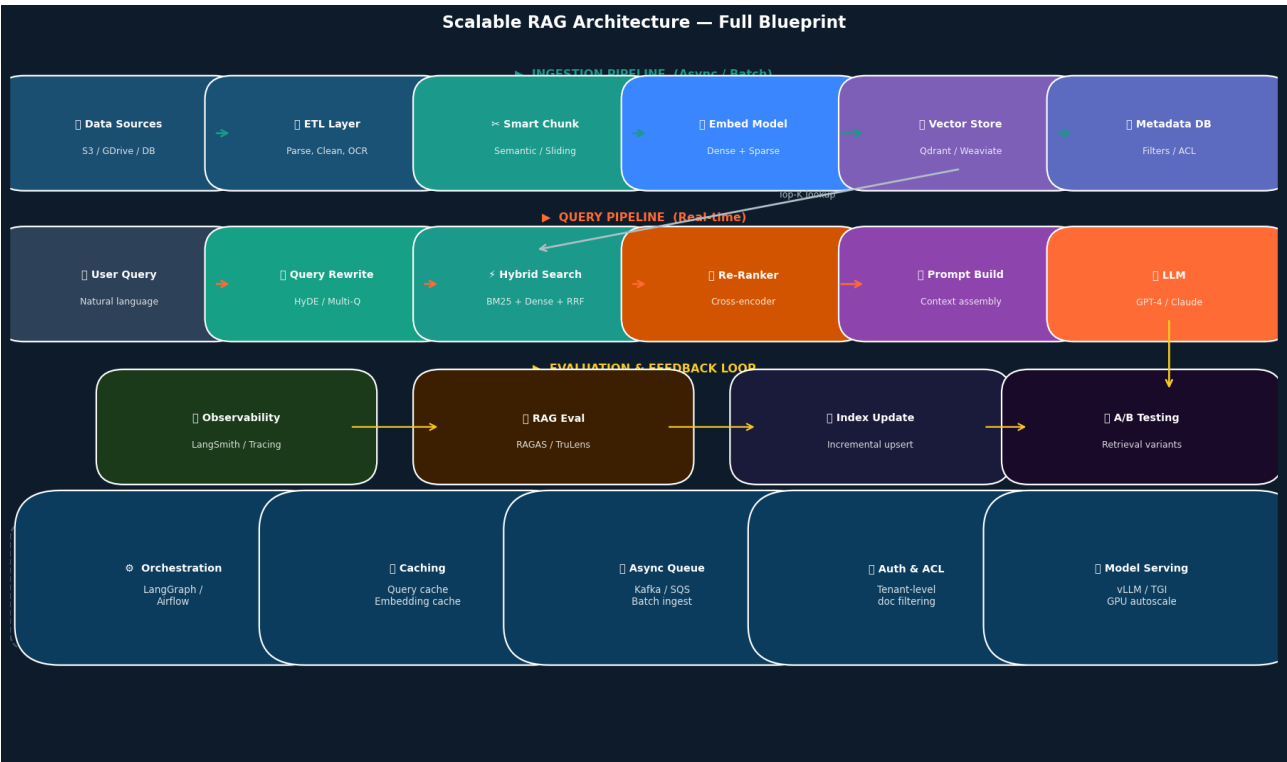


Figure 4: The complete Scalable RAG blueprint — ingestion, query, evaluation, and infra layers.

Section 5 — Key Components Deep Dive

5.1 Chunking Strategies

How you split documents is arguably the single biggest lever on retrieval quality. Chunks too large → noisy context, token limit overflow. Chunks too small → loss of surrounding context, split sentences mid-thought.

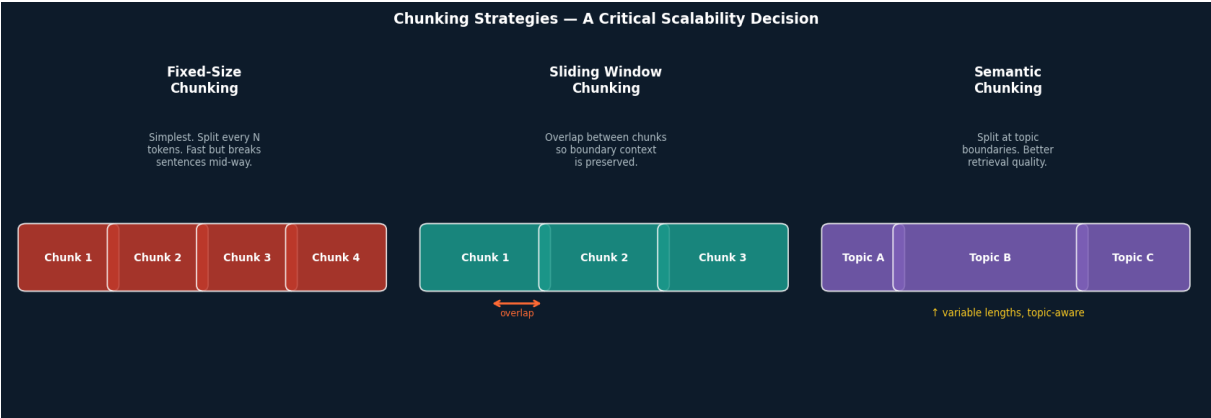


Figure 5: Fixed-size vs Sliding Window vs Semantic chunking — trade-offs at a glance.

Strategy	Chunk Size	Overlap	Best For	Tool
Fixed-Size	256–512 tok	None/small	Quick prototypes	LangChain CharacterSplitter
Sliding Window	256–512 tok	50–100 tok	Dense prose docs	LangChain RecursiveCharacter
Semantic	Variable	None	Mixed / long docs	LlamaIndex SemanticSplitter
Hierarchical	Chunk + Parent	N/A	Multi-granularity Q	LlamaIndex ParentRetriever

5.2 Retrieval Methods — Sparse, Dense, Hybrid

Retrieval is the heart of RAG. The quality of what you retrieve directly determines the quality of the final answer — 'garbage in, garbage out' applies doubly here.

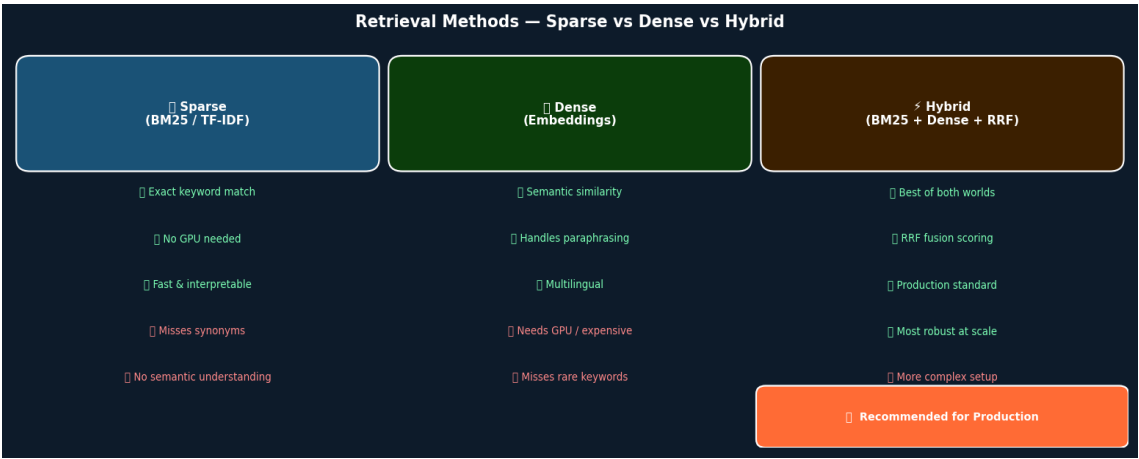


Figure 6: Retrieval method comparison. Hybrid (BM25 + Dense + RRF) is the production standard.

5.3 Query Intelligence

- **Query Rewriting:** Before embedding the query, rephrase it to improve recall. Technique: *HyDE (Hypothetical Document Embeddings)* — ask the LLM to generate a hypothetical answer, embed that, and use it to retrieve.

- **Multi-Query Retrieval:** Generate 3–5 variations of the same query, retrieve for each, and deduplicate. Dramatically improves recall for ambiguous questions.
- **Sub-question Decomposition:** For complex queries like *Compare Zomato's and Swiggy's delivery SLA policies*, break into two sub-questions, retrieve independently, then synthesise.

5.4 Re-Ranking

Initial retrieval returns Top-K candidates. A **cross-encoder re-ranker** (e.g. ms-marco-MiniLM-L-12-v2, Cohere Rerank API) then scores each candidate *in relation to the query* — much more accurate than cosine similarity alone. Re-ranking a Top-20 pool down to Top-3 often doubles answer accuracy. Cost: ~50ms extra latency. Worth it.

Section 6 — Production Checklist & What to Expect in Class

Before shipping a RAG system to production, walk through this checklist. In class, we'll build and evaluate each of these components hands-on.

■ Ingestion

- Multi-source connectors (S3, GDrive, Confluence, SQL)
- Robust parsing: PDFs with tables, scanned docs (OCR), HTML
- Metadata tagging: source, date, author, access_level
- Incremental upsert — only re-embed changed docs

➤ ■ Chunking

- Semantic or hierarchical chunking (not just fixed-size)
- Chunk size tuned per domain (legal: larger; FAQs: smaller)
- Parent-child retrieval for multi-granularity

■ Retrieval

- Hybrid retrieval (BM25 + dense) with RRF fusion
- Metadata pre-filtering (date range, department, access level)
- Re-ranking pass with cross-encoder
- Query rewriting / HyDE for ambiguous queries

■ Generation

- Structured prompt template with clear instructions + context block
- Citation injection — link each claim to a source chunk
- Guardrails — refuse out-of-scope questions cleanly

■ Evaluation

- RAGAS metrics: Context Precision, Context Recall, Faithfulness, Answer Relevancy
- LLM-as-judge for qualitative assessment
- A/B test retrieval variants before promoting to prod

■ ■ Infra

- Query-level caching (same question → cache hit)
- Embedding-level caching (same chunk → skip re-embed)
- Async ingestion queue (Kafka / SQS) for large data volumes
- Tenant-level ACL: user should not retrieve other user's docs

What We'll Build in Class

- ■ Hands-on: Build a scalable RAG pipeline from scratch using LangChain + Qdrant
- ■ Live demo: Hybrid retrieval vs dense-only — side-by-side quality comparison
- ■ Lab: Evaluate your RAG system using RAGAS on a real dataset
- ■ ■ Design exercise: Architect a multi-tenant RAG system for a B2B SaaS product

- ■ Deep dive: Query rewriting, HyDE, and re-ranking — when and how to use them

■ **Come Prepared With:** *A basic understanding of vector embeddings (cosine similarity, what embedding models do), familiarity with Python and the LangChain or LlamaIndex library, and a question about a RAG system you've either built or want to build. The best sessions are the ones where you bring a real problem to the table.*